

Software Requirements Specification (SRS) Template

This Software Requirements Specification (SRS) template is structured according to the ISO/IEC/IEEE 29148:2018 standard (which supersedes IEEE 830) ¹. It includes all key sections and provides example content and placeholders that can be adapted for any software project. Sections are numbered and titled for clarity, covering Introduction, Referenced Documents, Overall Description, Specific Requirements, and Appendices.

1. Introduction

1.1 Purpose

The purpose of this SRS is to describe the functional and non-functional requirements for the [SystemName] project. It defines the expected behavior, features, and constraints of the system to guide the development team and inform stakeholders. This document serves as an agreement between stakeholders and developers on what the system will deliver ².

1.2 Scope

The scope outlines the boundaries of the [SystemName] system. For example:

- **In Scope:** Core functionality such as user account management, data processing, and reporting features.
- **Out of Scope:** Features like hardware device control or third-party social media integration (if not needed).

This section clarifies what the system will and will not do. For instance, "[SystemName] will provide secure online data storage and retrieval for authenticated users; it will **not** include built-in email or messaging services." Describing scope helps prevent misunderstandings about system boundaries ³.

1.3 Definitions, Acronyms, and Abbreviations

Provide definitions of terms and acronyms used in this SRS for clarity ⁴. For example:

- **SystemName:** A placeholder name for the target software system.
- **User:** A person who interacts with [SystemName] (e.g., administrator or end user).
- **Administrator:** A user with elevated privileges to manage system settings.
- **Functional Requirement:** A requirement that specifies a function the system must perform.
- **Non-Functional Requirement:** A requirement that specifies a quality attribute (performance, security, etc.).
- **API:** Application Programming Interface – a set of methods for communication between software components.
- **GUI:** Graphical User Interface – the visual interface presented to users.
- **SRS:** Software Requirements Specification – this document.

(Include as many definitions and acronyms as needed for your project.)

2. Referenced Documents

List any external documents, standards, or references cited in this SRS ⁵. Include version or publication details. For example:

- **ISO/IEC/IEEE 29148:2018** – Systems and software engineering – Life cycle processes – Requirements engineering (SRS standard).
- **IEEE Std. 830-1998** – IEEE Recommended Practice for Software Requirements Specifications.
- **IEEE Std. 730-2014** – Software Quality Assurance Plans.
- **[Company XYZ] Design Guidelines** – Internal design standards for [Company].
- **User Manual – [SystemName] v1.0** – Preliminary user manual (if available).

These referenced documents provide background or complementary information to this SRS.

3. Overall Description

This section gives the high-level context for the system (the “big picture”) ⁶ ⁷.

3.1 System or Product Perspective

Describe how **[SystemName]** fits into the larger environment. For example, “The [SystemName] is a standalone web application that interfaces with the company’s central database. It integrates with an authentication service for user login and uses a third-party payment gateway for processing transactions.” If applicable, include a context diagram showing external systems, users, and data flows. Mention any operating platforms: e.g., “The system is designed to run on standard web browsers (Chrome, Firefox, Edge) on Windows and MacOS.”

3.2 System Functions

Summarize the major functions or features that **[SystemName]** will provide ⁸. For example:

- **User Management:** Allow users to register, log in, update profiles, and manage credentials.
- **Data Processing:** Perform data entry, validation, and calculations for [domain-specific data].
- **Reporting:** Generate reports (e.g., sales, usage statistics) in PDF or CSV format.
- **Search:** Enable users to search and filter records by various criteria.
- **Notifications:** Send email or on-screen alerts for important events.

This subsection gives a functional overview; detailed requirements are in Section 4.

3.3 User Characteristics

Describe the expected users of the system, their roles, and skill levels ⁹. For example:

- **End Users:** Non-technical personnel who will use the system for their daily tasks. They need an intuitive GUI with straightforward workflows.
- **Administrators:** Technical staff responsible for system configuration and maintenance; they require advanced privileges and detailed interfaces.

- **External Systems/Users:** If applicable, external system components (e.g., other software or services) that interact with [SystemName].

List any assumptions about user backgrounds. For instance, “Users are assumed to have basic computer literacy (able to use web browsers and email).”

3.4 Constraints, Assumptions, and Dependencies

Outline any constraints that affect system design, assumptions made, or dependencies on external factors ¹⁰. Examples include:

- **Regulatory Constraint:** The system must comply with GDPR data protection rules.
- **Technological Constraint:** Must operate on specified hardware/OS (e.g., Windows Server 2019, Linux).
- **Assumption:** A secure network connection is available at all times.
- **Dependency:** The system depends on an external authentication service (e.g., LDAP or OAuth provider).
- **Dependency:** Uses a cloud database service; the project assumes that required licenses will be procured.

Documenting these elements helps clarify the project’s context and any limitations.

3.5 Operating Environment

Describe the environment in which the system will operate ¹¹. For example:

- **Hardware:** Standard web servers (Intel/AMD 64-bit), 8GB RAM minimum.
- **Software:** Java 11+, Apache Tomcat 9+, MySQL 8.0 (or equivalent).
- **Network:** Ethernet/LAN with at least 100 Mbps connectivity; the application will communicate over HTTPS.
- **Other:** The system will be deployed on a cloud platform (AWS/Azure) and should support mobile browsers for selected features.

Mention any environmental conditions like temperature or physical location if relevant (e.g., for embedded systems).

4. Specific Requirements

This section details the **functional** and **non-functional** requirements. Each requirement should be clear, unambiguous, and testable ¹².

4.1 Functional Requirements

Functional requirements describe specific behaviors or functions ¹². Below are sample requirement statements (use your project’s context to replace placeholders):

- **[F1] User Login:** The system shall allow a registered user to log in using a unique username and password. Upon successful authentication, the user shall be directed to the main dashboard.
- **[F2] Account Management:** The system shall enable users to create, view, update, and delete their account profiles. Profile updates shall be saved to the database.
- **[F3] Data Search:** The system shall allow users to search records by multiple fields (e.g., date range, keywords). Search results shall be displayed in a paginated list sorted by relevance.

- **[F4] Report Generation:** The system shall provide functionality to generate a summary report of user activity. The report shall be available for download as a PDF file.
- **[F5] Audit Logging:** The system shall record a timestamped audit log entry for each user login and each record modification.
- **[F6] Error Handling:** The system shall validate all input fields and display user-friendly error messages for invalid data.

(Add or modify requirements to fit your system. Label each with a unique identifier for traceability ¹².)

4.2 Non-Functional Requirements

Non-functional requirements specify system qualities. Typical categories include performance, security, reliability, usability, maintainability, etc ¹³. Examples:

- **Performance:** The system shall support up to 100 simultaneous users with a page response time under 3 seconds. It shall process 500 transactions per minute during peak load.
- **Security:** All data in transit must be encrypted using TLS 1.2 or higher. User passwords shall be stored hashed with SHA-256. The system shall implement role-based access control, ensuring that users can only access permitted functions.
- **Reliability & Availability:** The system shall be available 99.9% of the time, with failover to a backup server in case of failure. Any single point of failure must be mitigated.
- **Usability:** The user interface shall be consistent and intuitive. Web pages shall comply with accessibility standards (e.g., WCAG 2.1 Level AA). Training should require no more than 2 hours for a new user.
- **Maintainability:** The codebase shall follow the project's coding standards and be documented. Modules should be loosely coupled to allow independent updates. The system should log detailed error information for troubleshooting.
- **Portability:** The application shall run on Windows and Linux servers without code changes. Any platform-specific dependencies shall be clearly documented.
- **Backup & Recovery:** The system shall perform nightly backups of the database. In the event of data loss, recovery time shall not exceed 4 hours and data loss shall not exceed 1 hour of transactions.

(Include additional non-functional requirements specific to your project, organized by category as needed.)

4.3 External Interface Requirements

Specify requirements for system interfaces ¹⁴ ¹⁵. Categories include user interfaces, hardware, software, and communications:

- **User Interfaces:**
 - The system shall provide a web-based graphical user interface with responsive design for desktop and mobile browsers.
 - The main screens include: Login, Dashboard, Search, Data Entry Form, and Reports. Each screen shall include navigation menus and help text.
 - Input fields must be validated on the client and server side to prevent invalid data entry.
- **Hardware Interfaces:**

- No specialized hardware is required. The system will run on standard server hardware.
- If the system interfaces with hardware (e.g., barcode scanners), specify port and protocol details (e.g., USB keyboard-emulation). (Placeholder for projects needing hardware interfaces.)

- **Software Interfaces:**

- The system shall expose RESTful APIs for data access (e.g., JSON over HTTPS). API endpoints will be documented with Swagger/OpenAPI.
- It shall integrate with the company's existing Customer Relationship Management (CRM) system via SOAP or REST API (specify endpoints and data formats).
- The database interface shall use JDBC (for Java) or an ORM (e.g., Entity Framework) to connect to the chosen RDBMS.

- **Communications Interfaces:**

- The system shall use HTTPS (TCP port 443) for all client-server communications.
- Email notifications shall use SMTP to a designated mail server.
- If relevant, specify any message queue or data bus (e.g., "Integrate with Kafka on port 9092 for event streaming").

Each interface requirement should detail the interacting components and data formats.

4.4 Use Cases

Use cases (or user scenarios) illustrate typical interactions with the system. Example use cases include:

- **Use Case: User Authentication** – *Actors:* Registered User. *Description:* The user enters username and password on the Login screen. The system validates credentials and grants access to the Dashboard. On failure, an error message is shown. (*See Appendix B for a detailed sequence diagram.*)
- **Use Case: Generate Activity Report** – *Actors:* Administrator. *Description:* The admin selects a date range and report type on the Reports page. The system compiles relevant data and presents a summary. The admin can download the report as PDF. If no data is found, the system indicates "No records available."*

(Additional use cases should be documented as needed, capturing the flow of events and alternate paths.)

5. Appendices

Appendices provide supplementary information. They are optional but can include items such as a glossary, detailed use cases, or a traceability matrix ¹⁶.

5.1 Appendix A: Glossary

Define any specialized terms or jargon used in this document (extended from Section 1.3). For example:

- **Session Timeout:** The duration of inactivity after which the user is automatically logged out.
- **REST API:** An application programming interface that uses HTTP requests to GET, PUT, POST, and DELETE data.
- **Sandbox Environment:** A test environment isolated from production where new features are trialed.

5.2 Appendix B: Use Case Details

Provide detailed descriptions or diagrams of use cases mentioned in Section 4. For example:

- **Use Case: User Login**
 - *Primary Actor:* End User
 - *Precondition:* The user must be registered and have a username/password.
 - *Main Flow:*
 1. User navigates to the login page.
 2. User enters username and password.
 3. System validates credentials.
 4. If valid, system establishes a session and redirects user to the Dashboard.
 - *Alternate Flow (Invalid Credentials):*
 - 3a. If credentials are incorrect, system displays an error message and prompts for retry.
 - *Postcondition:* User is authenticated and in the system (or remains on login page after failure).

(Additional use cases can be documented similarly.)

5.3 Appendix C: Traceability Matrix

A traceability matrix maps requirements to design elements and test cases. For example:

Requirement ID	Description	Linked Test Case ID	Design Module
F1	User login functionality	TC-Login-01	AuthModule
F3	Search records functionality	TC-Search-02	SearchService
NFR-Sec-01	TLS encryption for data	TC-Sec-Encryption	SecurityLayer

(This table is illustrative; expand it to cover all requirements.) ¹⁷

(Additional appendices can include data dictionaries, mockup diagrams, or anything relevant.)

References: This template is informed by IEEE/ISO 29148:2018 recommendations ¹ and similar SRS guidelines ¹⁸ ¹². It should be tailored to match the specific needs and context of your project.

1 ISO/IEC/IEEE 29148 Requirements Specification Templates | ReqView Documentation

<https://www.reqview.com/doc/iso-iec-ieee-29148-templates/>

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 ISO/IEC/IEEE 29148 Systems and Software Requirements Specification (SRS) Example Template

<https://www.well-architected-guide.com/documents/iso-iec-ieee-29148-template/>